Phase 1 – Baseline Chat Application (No Security)

Protocol

The assignment contains a list of commands and their functionality which are to be parsed at client side, and sent to server in a pre-defined format understood by both parties. In this assignment, we use codes like LOGIN, MSG, FROM, EXIT, OK, ERROR, etc based on which the payload is handled.

The pipe operator '|' is used as a delimiter and after the parsing step is done at client side, the message is serialized in proper format, to prepare it to send to the server. The above process is best understood by example:

Scenario 1:

The *Server* is running on IP_ADDR:PORT, and a client *Alice* wants to login to the server.

```
> login Alice
```

At client side, the message is parsed as a login command, validity check for proper syntax, and then serialized into proper format, to send to server.

```
LOGIN|Alice
```

Scenario 2:

Client *Alice* wants to send <*message*> to client *Bob*, considering both are currently online.

```
>@Bob <message>
```

At client side, the message is parsed as a chat command to be sent to another client via server, validity checks are performed for proper syntax, and then serialized into proper format.

```
MSG|Bob|<message>
```

At server side, this message is received, and server checks in the client registry for socket file descriptor of receiver (in this case, Bob) and forwards message to them,

```
FROM|Alice|<message>
```

List of all Protocol message formats:

1. LOGIN|<username>
2. MSG|<receiver>|<message>
3. WHO
4. QUIT
5. OK|<message>
6. ERROR|<reason>
7. USERS|<username1>|<username2>...
8. FROM|<sender>|<receiver>

Message Framing

In this chat application, since client and server uses TCP connection to exchange messages, and TCP being a stream-oriented protocol, a mechanism for indicating message completion is required. The approach we followed is transmitting every payload with a "length-prefixed frame.

[length][payload]

Where length is a fixed 4-bytes unsigned integer which represents the number of bytes in the payload. The receiver would first get this 4-byte length and then read exactly that many bytes of payload, hence it is guaranteed that the receiver would retrieve complete messages from the buffer stream.

Client Registry

In this chat application, the server maintains a client registry. In the server, whenever a new client starts a connection, a map of client username to its socket file descriptor is stored. The client registry is then used to relay messages to the appropriate recipient of the message.

Suppose Alice and Bob are two clients already registered in the repository, Alice wants to send Bob the message "Hello". As discussed in the protocol part, the message will be serialized and sent to the server with the receiver as Bob. At server side, it queries the map and searches for username Bob to get its socket file descriptor so Alice's message is appropriately delivered. This is how server routes messages if more than two clients are connected.

Wireshark Screenshots

As no security mechanisms are put into place currently in the chat application, the serialized messages sent over the socket are visible as plaintext.

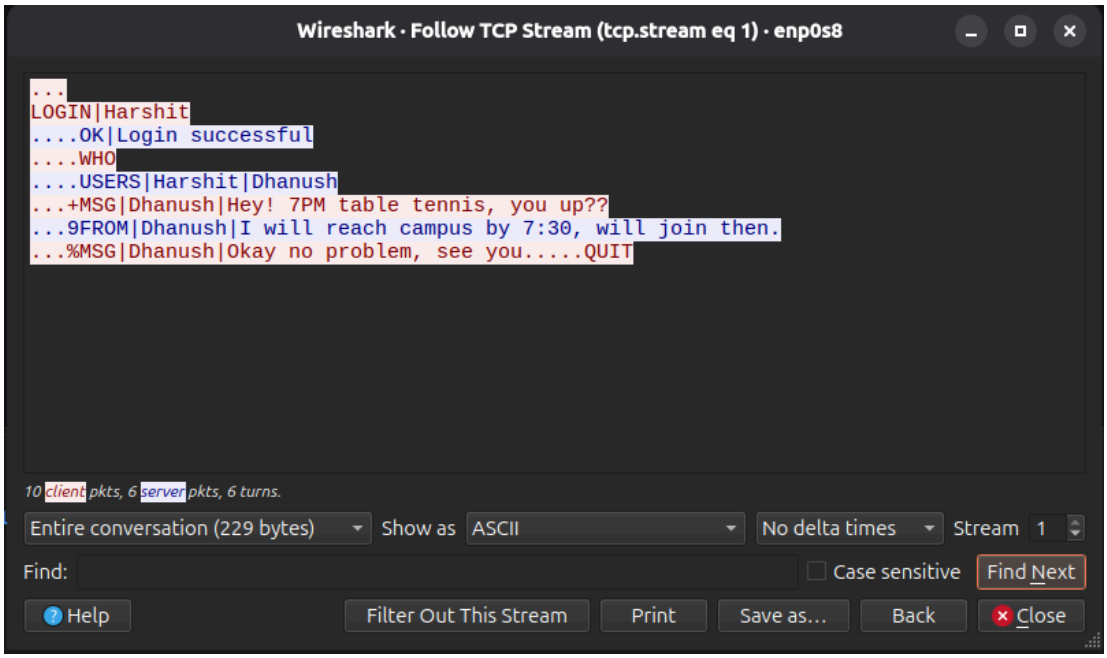

Fig 1.1 Client1 serialized messages

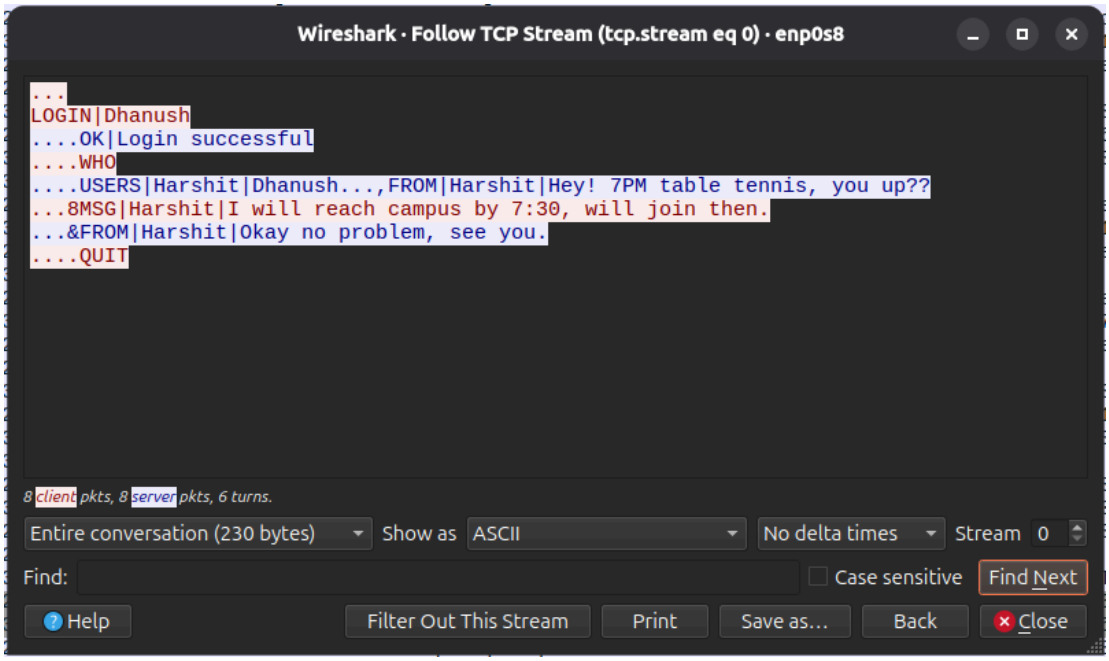

Fig 1.2 Client2 serialized messages

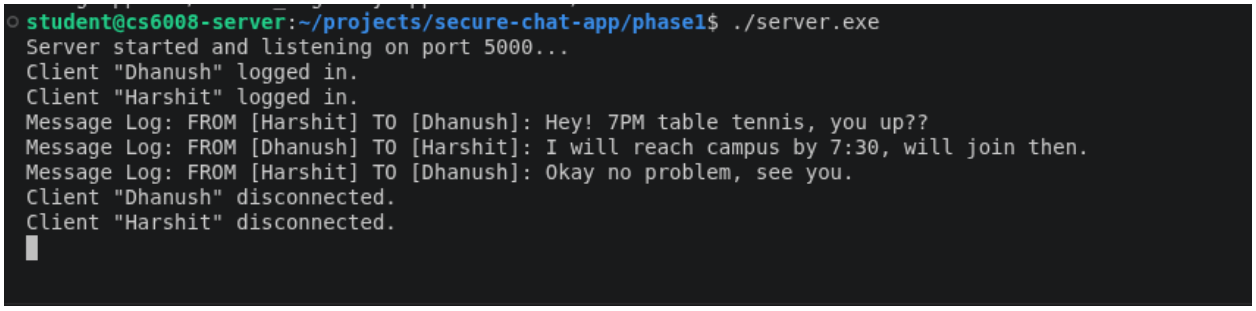

Fig 1.3 Server Logs

Phase 2 – Client-Server Confidentiality via Diffie–Hellman

Diffie–Hellman Key Exchange:

Prime and base are taken from RFC 3526 MODP group 14.

Client have a private_key : random BigNum of 256 bits

Server also have private_key : random BigNum of 256 bits

Calculations

Public_key = *g^(private_key) mod p*;

Secret_key = *(received_public_key)^(private_key) mod p*;

Steps

Client sends: DH_HELLO|client's public_key

Server receives DH_HELLO and calculates secret_key.

Server sends: DH_HELLO|server's public_key

Client receives DH_HELLO and calculates secret_key.

Secret_key achieved on both client and server.

Calculations

Aes_key = first 16 byte [SHA_256(Secret_key)]

DH Shared Secret

↓

SHA-256

↓

32-byte digest

↓ (take first 16bytes)

16-byte AES key

Hashing provides a fixed-size key suitable for AES and avoids directly using the raw DH integer as a symmetric encryption key.

Verify fingerprint using the function call: *print(SHA_256(aes_key))*

Encrypt text using the function call: *aes_gc,_encrypt(key, plaintext)*

The encrypted data is transmitted as format: *[IV | Ciphertext | Authentication Tag]*

Wireshark Screenshots

Both endpoints calculate and display a hash-based fingerprint of their derived shared secret. Matching fingerprints demonstrate that the client and server independently calculated the same DH shared secret. The raw shared secret and AES key are not displayed.

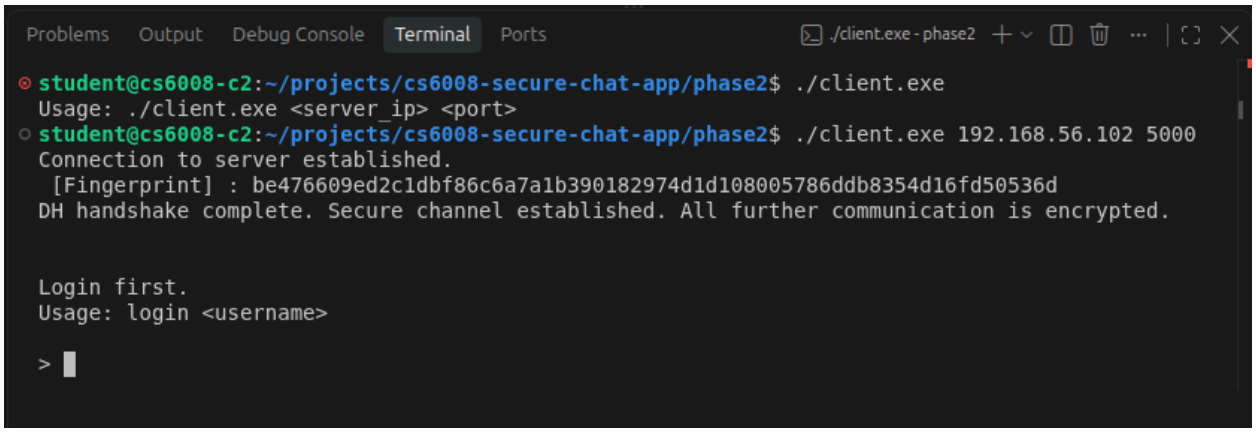

Fig 2.1 Fingerprint Client1 Side

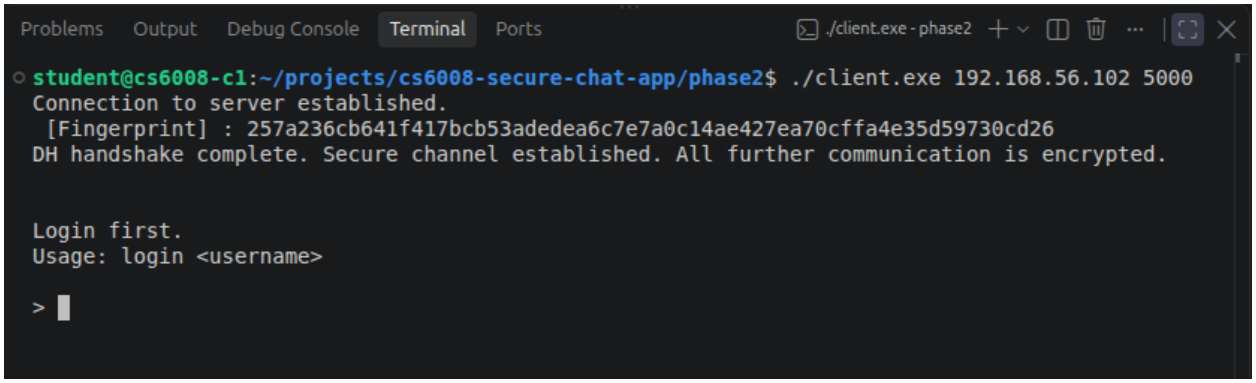

Fig 2.2 Fingerprint Client2 Side

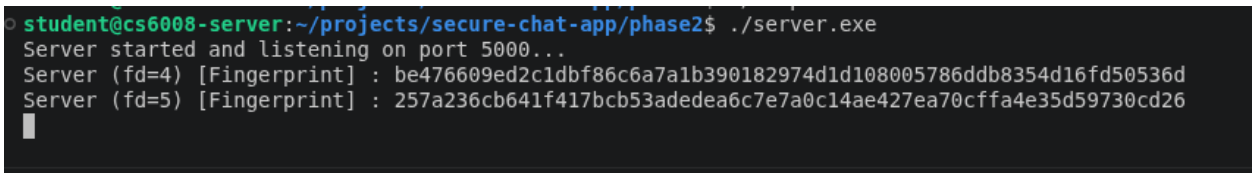

Fig 2.3 Fingerprint Server Side

A Wireshark capture was performed after implementing Phase 2. Unlike Phase 1, where the TCP stream exposed the plaintext message, the Phase 2 capture contains encrypted ciphertext and the original chat content is no longer readable.

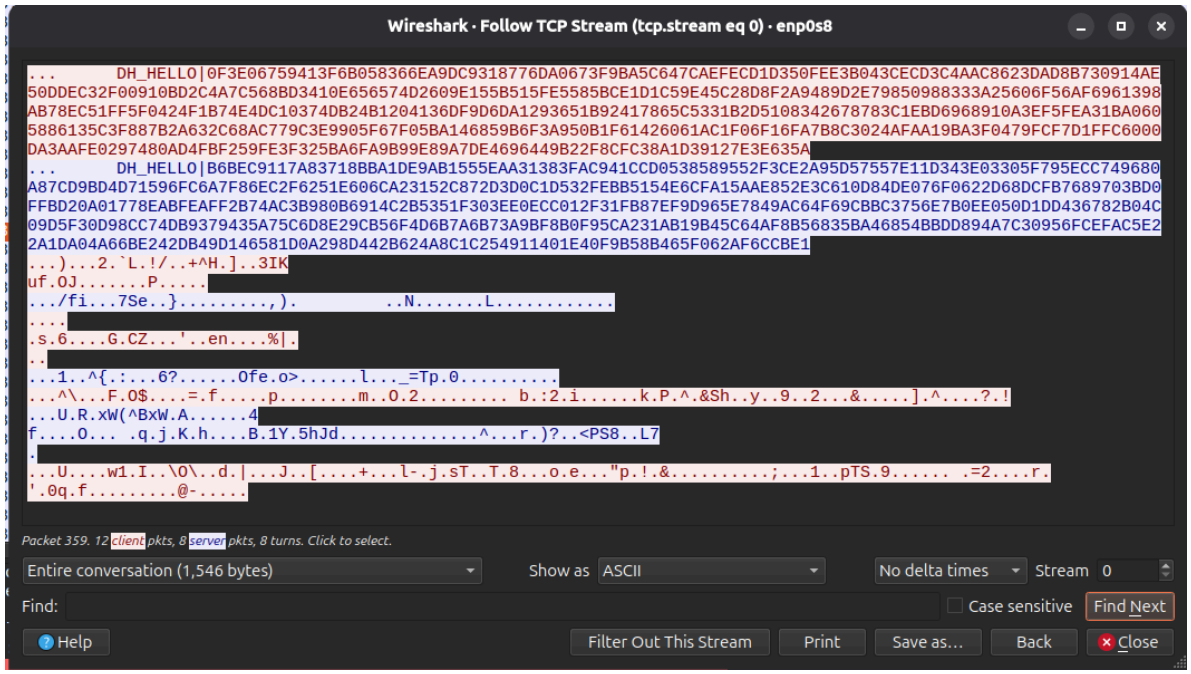

Fig 2.4 WireShark SS from client1 machine

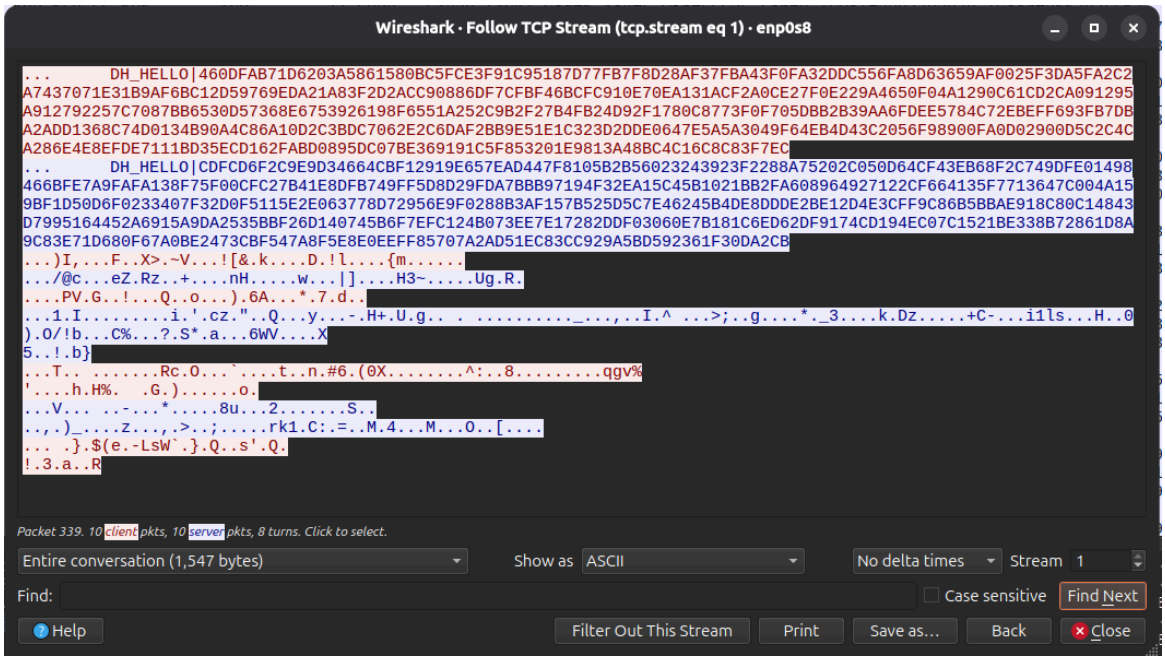

Fig 2.5 WireShark SS from client2 machine

AES-GCM Tamper Detection:

A captured ciphertext was modified by changing a single byte and then supplied to the receiving client. AES-GCM authentication detected the modification and rejected the message instead of producing corrupted plaintext. This demonstrates the integrity protection provided by the authentication tag.

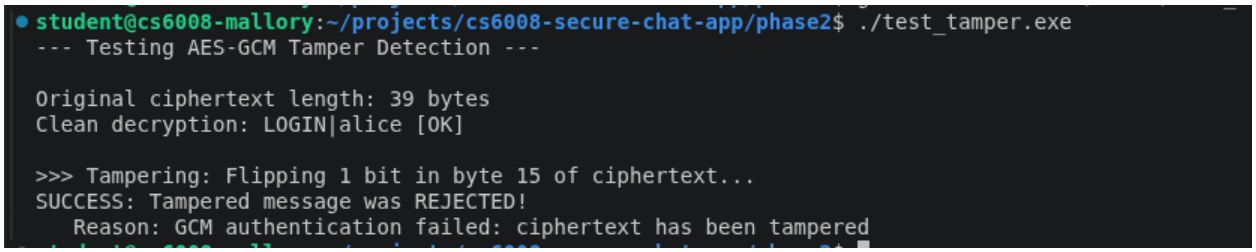

Fig 2.6 Tamper detection test

Man-in-the-Middle Attack:

Although Diffie–Hellman provides a shared secret, it does not authenticate the identity of the peer. Therefore, a MITM attacker can establish two independent DH exchanges: one with the client while pretending to be the server, and another with the server while pretending to be the client.

[Client C1] ← DH/K1 → [Mallory] ← DH/K2 → [Server]

Mallory decrypts messages received from the client using the first shared key, reads the plaintext, and then encrypts and forwards the message to the server using the second shared key. The same process is performed for messages travelling in the opposite direction.

The MITM proxy was executed on the separate Mallory VM as required. The proxy successfully intercepted and logged plaintext messages even though the client and server believed that they had established a secure connection.

```
Server (fd=6) [Fingerprint] : 0536d48b6c6e2f2b7c212f3c069ade786f0519d3e284066ed1a3ad61406298d2
Server (fd=4) [Fingerprint] : e91c41a4db41ae2acfca3001cc38bf4dc290748dd635cf0a319d5fc3c9a00a1c
Client "Harshit" logged in.
Client "Dhanush" logged in.
Message Log: FROM [Harshit] TO [Dhanush]: Yo! Just wanted to let you know, practice is 7 AM tomorrow.
Message Log: FROM [Dhanush] TO [Harshit]: Thanks for informing! I will make sure to arrive on time.
Message Log: FROM [Harshit] TO [Dhanush]: No problem.
Client "Harshit" disconnected.
Client "Dhanush" disconnected.
```

Fig 2.7 Server Logs

```
student@cs6008-mallory:~/projects/cs6008-secure-chat-app/phase2$ ./mitm.exe 192.168.56.102 5000
[MITM] Mallory proxy listening on port 5001
[MITM] Forwarding to real server at 192.168.56.102:5000

[MITM] Victim client connected (fd=4)
[MITM] Connected to real server at 192.168.56.102:5000
[MITM] Client<->Proxy fingerprint [Fingerprint] : d1225b2c7376d259e1bed227657fbdbcac4b0728188ef39756b495b2cc30796a
[MITM] Proxy<->Server fingerprint [Fingerprint] : 0536d48b6c6e2f2b7c212f3c069ade786f0519d3e284066ed1a3ad61406298d2
[MITM] Both DH handshakes complete — intercepting all traffic.

[MITM] C->S: LOGIN|Harshit
[MITM] S->C: OK|Login successful
[MITM] C->S: WHO
[MITM] S->C: USERS|Dhanush|Harshit
[MITM] C->S: MSG|Dhanush|Yo! Just wanted to let you know, practice is 7 AM tomorrow.
[MITM] S->C: FROM|Dhanush|Thanks for informing! I will make sure to arrive on time.
[MITM] C->S: MSG|Dhanush|No problem.
[MITM] C->S: QUIT
[MITM] Session (fd=4) ended.
```

Fig 2.8 Mallory Logs

Why the MITM Attack Succeeds:

The attack succeeds because the Phase 2 DH exchange does not authenticate the party providing the public DH value. The client can verify that a DH exchange completed successfully, but it cannot determine whether the other endpoint is the legitimate server or Mallory.

List of Protocol message format added in this step:

1. DH_HELLO|<Public_key>

Phase 3 – Server Authentication via PKI

OpenSSL commands to setup CA and issue the server certificate

Command	Description
openssl genrsa \ -out ca/ca.key 2048	Generate a 2048-bit RSA private key for Certificate Authority(CA)
openssl rsa -in ca/ca.key \ -text -noout	Display CA private key parameters in human readable form
openssl req -x509 -new -nodes \ -key ca/ca.key \ -sha256 \ -days 365 \ -out ca/ca.crt	Creates a self-signed CA certificate using CA's private key. The certificate uses SHA-256 and its validity is 365 days.
openssl x509 -in ca/ca.crt \ -text -noout	Displays the CA certificate details
openssl genrsa \ -out server/server.key 2048	Generates a 2048-bit RSA private key for the server
openssl req -new \ -key server/server.key \ -out server/server.csr	Creates a Certificate Signing Request(CSR) containing server's identity information and public key, which is submitted to CA for signing.
openssl x509 -req \ -in server/server.csr \ -CA ca/ca.crt \ -CAkey ca/ca.key \ -CAcreateserial \ -out server/server.crt \ -days 365 -sha256	Uses the CA's private key to sign the server CSR, which produces the server certificate. The certificate uses SHA-256 and its validity is 365 days.
openssl x509 -in server/server.crt \ -text -noout	Display the issued server certificate details

Server Authentication

When a client sends a connection request to the server, now before any diffie-hellman exchange begins, teh server sends its X.509 certificate to the client. The client validates this certificate using the local copy of Certificate Authority's (CA) root certificate. The client verifies 3 things, (a) if the server certificate is signed by the trusted CA, (b) if the certificate is within its validity period, and (c) if the certificate Common Name (CN) matches the expected server identity. If any of these fails, client aborts, diffie-hellman key exchange never happens in that scenario.

Though, a certificate on its own doesn't prove that the party on the other side actually holds the matching private key. Hence after certificate validation, the client issues a random challenge (32 bytes) and sends it to the server. The server needs to sign this challenge with its private key and send the response back to the client. The client then verifies that signature using the public key of the server embedded in the certificate. So only after both certificate and signature are verified, the client moves on to continue with Diffie-Hellman and AES-GCM as happened in Phase2.

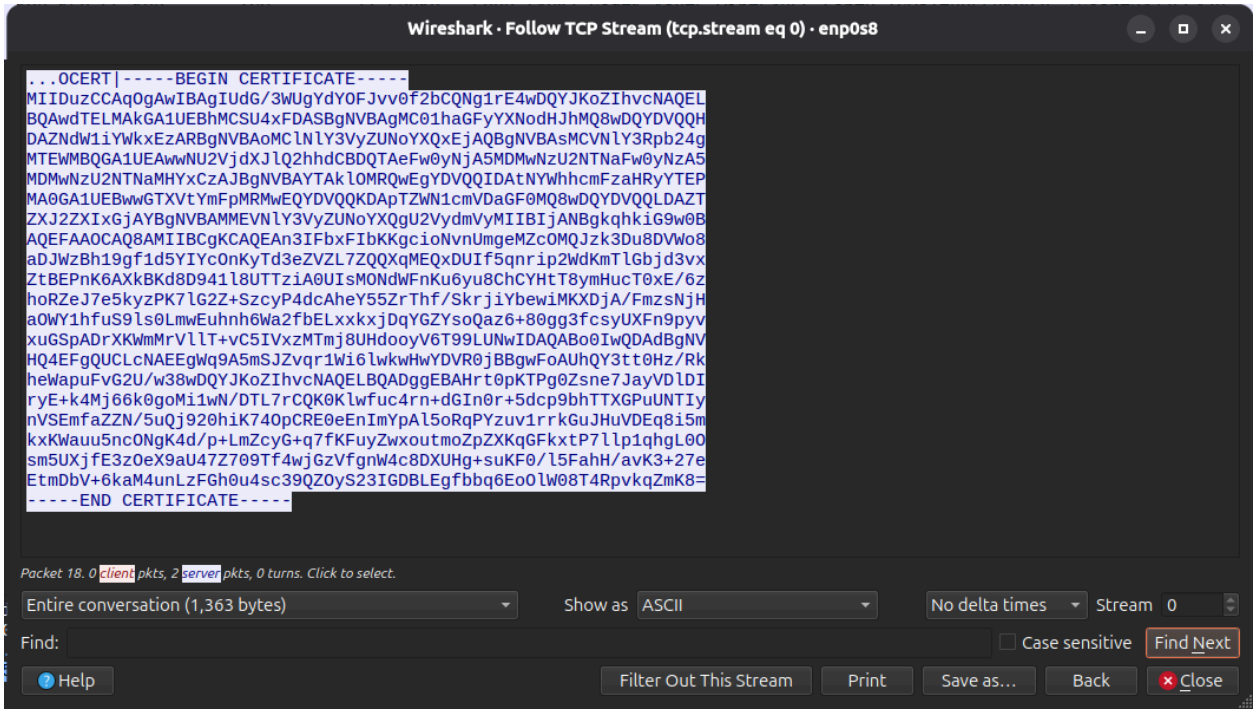

Fig 3.1 Server certificate sent when a client request connection

Why the Phase 2 MITM attack fails in Phase 3

In Phase 2, Mallory was able to sit between the client and server and perform two separate Diffie-Hellman exchanges, one with the client and one with the server. Since Diffie-Hellman by itself does not verify the identity of the other party, neither the client nor the server could detect that Mallory was in between them.

In Phase 3, this is handled using X.509 certificates. Before starting the Diffie-Hellman exchange, the client first asks the server for its certificate. The client then verifies the certificate using the trusted CA certificate stored locally. Since Mallory does not have the CA's private key, she cannot create a fake certificate that will be accepted by the client. Therefore, the MITM attack from Phase 2 fails before reaching the Diffie-Hellman step.

Testing the MITM attack

Case 1: Mallory uses a self-signed certificate

In this case, Mallory presents her own certificate (mallory.crt) to the client instead of the real server certificate. Since this certificate is not signed by the trusted CA, the certificate verification fails. As a result, verify_server_certificate() returns false and the client aborts the connection. No challenge is sent and the Diffie-Hellman exchange does not take place. This shows that the client is checking whether the certificate is actually trusted, and not just checking whether a certificate is present.

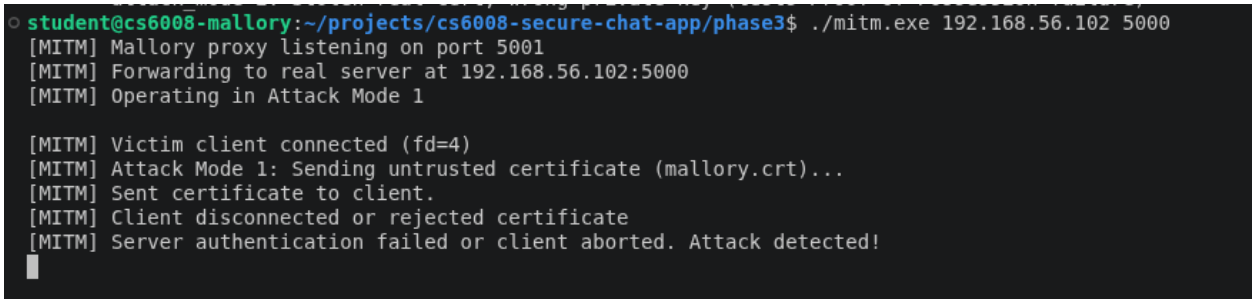

Fig 3.2 Mallory sends her own certificate to client

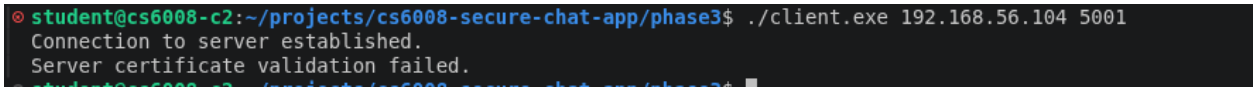

Fig 3.3 Client rejects the certificate

Case 2: Mallory uses the real server certificate but does not have the private key

In this case, Mallory has copied the real server.crt from the server and presents it to the client. Since the certificate is genuinely signed by the trusted CA, the certificate verification succeeds.

However, Mallory does not have the corresponding server.key. When the client sends the random challenge, Mallory tries to sign it using her own private key (mallory.key). The client then verifies the signature using the server's public key stored inside the certificate. Since the signature was created using a different private key, the verification fails and the client aborts the connection.

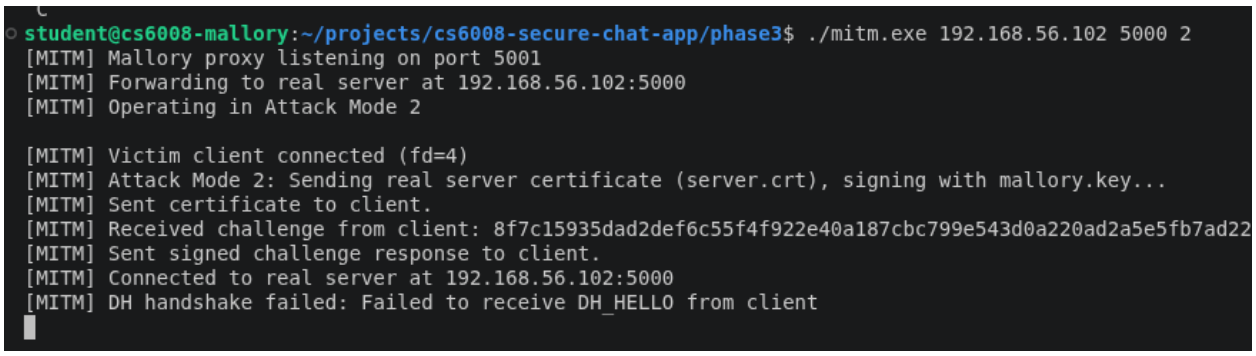

Fig 3.4 Mallory sends server's legitimate certificate to client

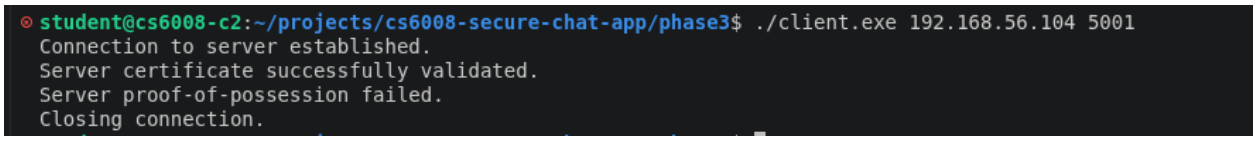

Fig 3.5 Client sends challenge to Mallory, which they fails

This shows why the challenge-response step is also required. Having a valid certificate is not enough; the party must also prove that it actually owns the private key corresponding to that certificate.

Therefore, compared to Phase 2, Phase 3 has two separate checks against the MITM attack. The certificate verification prevents Mallory from using an untrusted certificate, while the challenge-response mechanism prevents her from using a copied certificate without the corresponding private key. Only when both checks succeed does the client continue with the Diffie-Hellman key exchange and AES-GCM communication.

List of Protocol message format added in this step:

1. CERT|<PEM_certificate>
2. CHALLENGE|<hex_challenge>
3. CHALLENGE_RESP|<hex_signature>

Phase 4 – End-to-End Encryption (E2EE)

E2E Protocol Messages

The approach followed was to encapsulate the E2E protocol messages inside the standard MSG and FROM frames, detailed format as follows:

1) Handshake Init:

MSG|<receiver>|__E2E_INIT__|<client_public_key> //toserver

FROM|<sender>|__E2E_INIT__|<client_public_key> //fromserver

2) Handshake Acknowledgment:

MSG|<receiver>|__E2E_ACK__|<client_public_key>

FROM|<sender>|__E2E_ACK__|<client_public_key>

3) End-to-End Encrypted Message:

MSG|<receiver>|__E2E_MSG__|<hex_encoded_ciphertext>

FROM|<sender>|__E2E_MSG__|<hex_encoded_ciphertext>

Objective:

Establish an end-to-end cryptographic channel between two chat clients (C1 and C2) such that intermediate nodes (including the chat server itself) are incapable of inspecting, reading, or tampering with the plaintext chat messages. The server acts purely as an oblivious relay.

Peer-to-Peer Diffie–Hellman over Server Relay

DH Group: RFC 3526 MODP Group 14 (2048-bit prime p, base g=2).

Per-Session Keys: When C1 initiates an E2E session with C2, both generate fresh, ephemeral 256-bit BigNum private keys independent of the link-layer client-server keys.

Calculations:

Public_Key=g $^{private\_key}$(mod p)

Shared_Secret=(Peer_Public_Key) $^{private\_key}$(modp)

Derivation:

E2E_AES_Key=first 16 bytes of SHA-256(Shared_Secret)

E2E Handshake Steps:

1. Initiation (C1→C2 via Server):
    - User types */e2e <username>*.
    - C1 generates keypair (a,g^a(mod p)), stores it in its pending map, and sends: *MSG|C2|__E2E_INIT__|<hex_of_g^a>*
    - The message is encrypted under the C1 ↔Server AES key.
    - The server decrypts the outer frame, inspects routing information (receiver = C2), re-encrypts the frame under the Server ↔ C2 AES key, and forwards it to C2 as: *FROM|C1|__E2E_INIT__|<hex_of_g^a>*

2. Acknowledgment & Secret Computation (C2→C1 via Server):
    - C2 receives the __E2E_INIT__ frame, generates its own keypair (b, g^b mod p), and computes the shared secret: *Secret = (g^a)^b mod p*
    - C2 derives the 16-byte E2E_AES_Key and registers it in its active sessions map: *aes_keys[C1] = E2E_AES_Key.*
    - C2 sends its public key back:

      MSG|C1|__E2E_ACK__|<hex_of_g^b>
    - The server routes this message to C1 as:

      FROM|C2|__E2E_ACK__|<hex_of_g^b>

3. Session Finalization on C1:
    - C1 receives __E2E_ACK__, retrieves the stored keypair from pending[C2], and computes: *Secret = (g^b)^a mod p*
    - C1 derives the identical 16-byte E2E_AES_Key and stores it: *aes_keys[C2] = E2E_AES_Key.*
    - C1 erases the pending entry. The direct E2E channel is established.

Double Encryption (Outer Transport + Inner E2E):

Every chat message sent over an active E2E session undergoes two layers of authenticated encryption:

Plaintext Chat: "Hello Bob"

↓

[Layer 1: Inner E2E Encryption]

aes_gcm_encrypt(E2E_AES_Key, "Hello Bob")

Format: *[IV (12B) | Ciphertext | Tag (16B)]*

Hex-encoded string: <hex_blob>

Enclosed in frame: *MSG|bob|__E2E_MSG__|<hex_blob>*

↓

[Layer 2: Outer Link-Layer Encryption]

aes_gcm_encrypt(Client_Server_Key, Frame_Payload)

Format: *[4-byte length][IV (12B) | Ciphertext | Tag (16B)]*

↓

Transmitted over TCP Socket to Server

- When the server receives the frame, it strips only Layer 2. The server observes only *MSG|bob|__E2E_MSG__|<hex_blob>*, which contains no recognizable plaintext.
- The server forwards the payload wrapped in its link-layer key to the recipient.
- The recipient strips Layer 2, recognizes *__E2E_MSG__|*, and strips Layer 1 using *aes_keys[sender]*.

Verification Screenshots

1. Matching E2E Fingerprints

Both clients calculate and display the SHA-256 fingerprint of the derived E2E key:

$$Fingerprint = SHA\text{-}256(E2E\_AES\_Key)$$

Both clients print the identical fingerprint string, proving that C1 and C2 agreed on the same symmetric key without the server ever obtaining it.

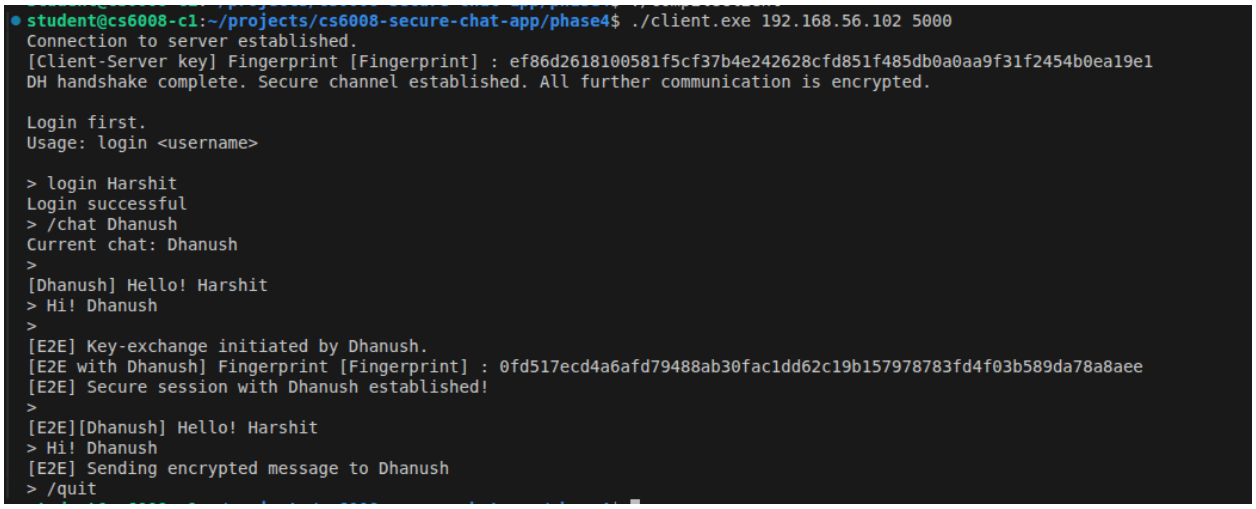

Fig 4.1 Client1 fingerprint logs

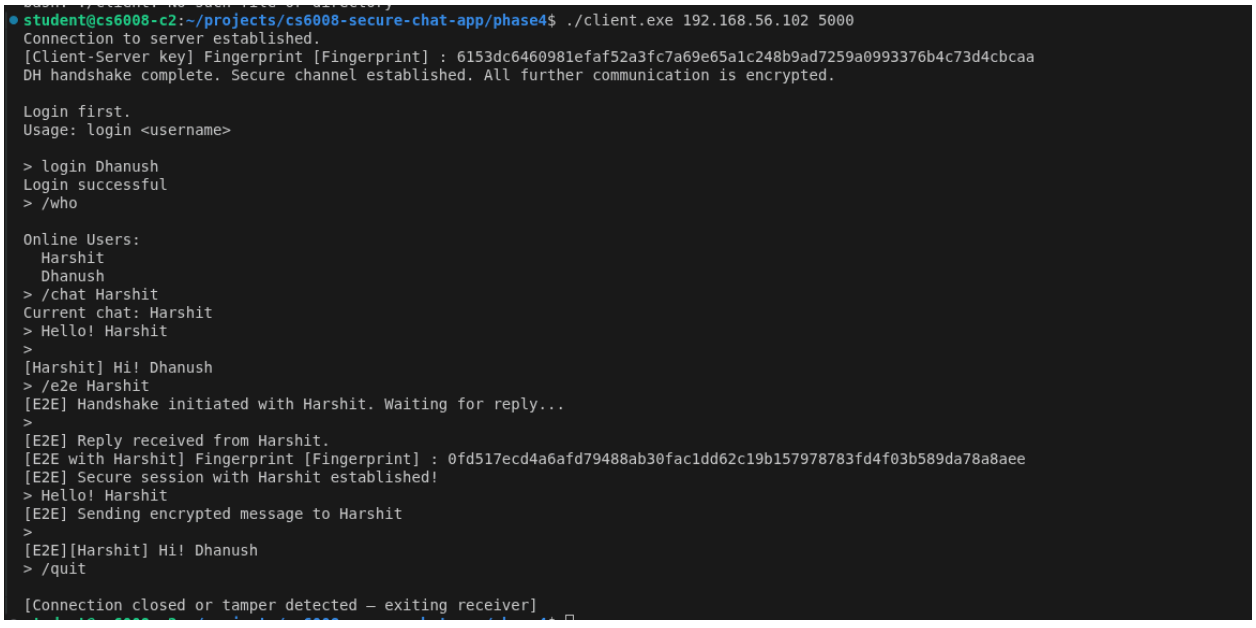

Fig 4.2 Client2 fingerprint logs

Sessions are maintained per-peer using std::map<std::string, std::vector<uint8_t>>. A client can engage in concurrent, independent E2E encrypted sessions with multiple users simultaneously without key collisions or session cross-talk.

2. Server-Side Eavesdropping Resistance

Server console logs were inspected during live message transmission. While the server displays the sender and recipient routing headers, the message content displayed in server stdout is an opaque hex string (__E2E_MSG__|...). The server is unable to inspect the conversation contents.

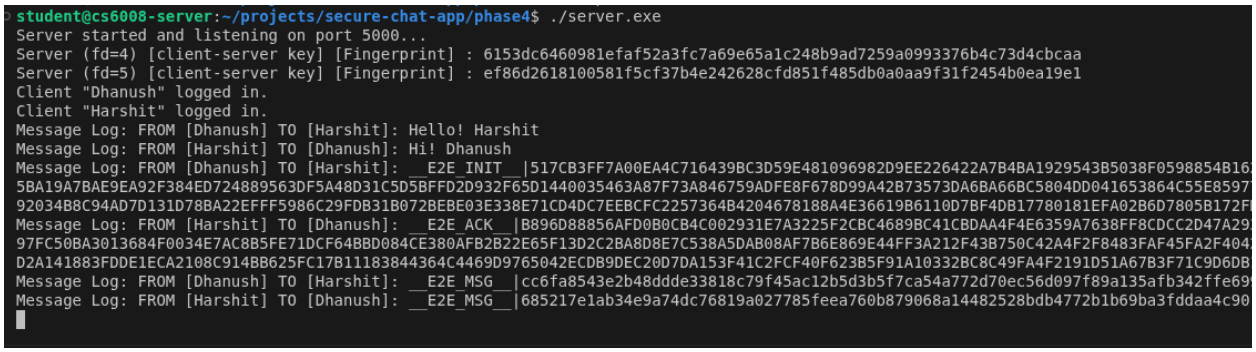

Fig 4.3 Server unable to read messages because encrypted

List of Protocol message format added in this step:

1. MSG|<receiver>|__E2E_INIT__|<new_ephemeral_public_key>
2. FROM|<sender>|__E2E_INIT__|<new_ephemeral_public_key>
3. MSG|<receiver>|__E2E_ACK__|<new_ephemeral_public_key>
4. FROM|<sender>|__E2E_ACK__|<new_ephemeral_public_key>
5. MSG|<receiver>|__E2E_MSG__|<hex_encoded_ciphertext>
6. FROM|<sender>|__E2E_MSG__|<hex_encoded_ciphertext>

Phase 5 – Forward Secrecy

Objective:

Ensure Forward Secrecy (PFS) on the end-to-end communication channel. Even if an adversary records all encrypted network traffic and subsequently compromises a client's active symmetric key or long-term credentials at time T, they cannot decrypt any conversation traffic exchanged prior to T.

Periodic Ephemeral DH Renegotiation (Rekeying)

1. Rekey Interval: Fixed timer firing every 60 seconds for all active E2E sessions.
2. Independence: Every rotation performs a brand-new Diffie–Hellman exchange with freshly generated random 256-bit BigNum private exponents. The new key is not derived from the old key (no key ratcheting/hash chaining); it is mathematically independent.
3. Immediate Discarding: Once the new key is confirmed, the previous session key is overwritten and removed from memory.

Collision Avoidance: Role-Based Tie-Breaking

The Problem:

Because both clients run their own independent 60-second timers, both endpoints would periodically attempt to trigger a rekey at approximately the same time. If both sides simultaneously sent an __E2E_INIT__ with new public keys, both would act as initiators, producing two conflicting, mismatched keys.

The Solution:

A deterministic lexicographic tie-breaking rule based on usernames:

Initiator = min(username_A, username_B)

Only the initiator will be responsible to refresh keys, this applies to each pair of clients for which e2e session is active.

Disruption-Free Chat Delivery:

- Chat messages continue to flow before, during, and after renegotiation.
- When an *__E2E_INIT__* is in transit, the sender and receiver continue decrypting and encrypting using their existing active key in aes_keys[peer].
- The in-progress DH keypair is staged in a separate map pending[peer].
- Once *__E2E_ACK__* is received, the atomic swap of aes_keys[peer] occurs under a mutex lock.
- Because the DH exchange completes in a matter of milliseconds, ongoing message delivery is uninterrupted.

Verification Screenshots

1. Fingerprint Evolution Across Rotations

Both clients logged the key fingerprint and system timestamp across multiple 60-second intervals.

A. Fingerprint Changes: The fingerprint string changes on every rotation, confirming a genuinely new key was generated.
B. Consistent Agreement: After each rotation, both clients print the exact same new fingerprint simultaneously.

Example log:

[Client 1 - Alice]

[REKEY] Sent renegotiation request to bob at 2026-09-04 18:33:00

[E2E with bob] [Fingerprint] : a1b2c3d4e5…

[REKEY] Sent renegotiation request to bob at 2026-09-04 18:34:00

[E2E with bob] [Fingerprint] : f9e8d7c6b5...

[Client 2 - Bob]

[E2E with alice] [Fingerprint] : a1b2c3d4e5... (Rotation 1 - matches Alice)

[E2E with alice] [Fingerprint] : f9e8d7c6b5... (Rotation 2 - matches Alice)

```
student@cs6008-c1:~/projects/cs6008-secure-chat-app/phase5$ ./client.exe 192.168.56.102 5000
Connection to server established.
Server certificate successfully validated.
proof-of-possession verified.
Server authentication complete.

[Client-Server key] Fingerprint [Fingerprint] : 1b712216ee1d5510bd5f3aa81eecb54296ebf8a7bc2294d1d4520ee34ef1bd10
DH handshake complete. Secure channel established. All further communication is encrypted.

Login first.
Usage: login <username>

> login Harshit
Login successful
> /chat Dhanush
Current chat: Dhanush
>
[E2E] Key-exchange initiated by Dhanush.
[E2E with Dhanush] Fingerprint [Fingerprint] : 6d589cfbc7058c7e0fc8edc79147ca6e80905eb5866c56d47d03c52abf6b901d
[E2E] Secure session with Dhanush established!
>
[E2E][Dhanush] Hello! How are you doing??
>
[E2E] Key-exchange initiated by Dhanush.
[E2E with Dhanush] Fingerprint [Fingerprint] : a47e43c65b3f6c6ff675b901c5d6975a3de45eabc525d88bf5efc7b2a3f3d3fd
[E2E] Secure session with Dhanush established!
>
[E2E] Key-exchange initiated by Dhanush.
[E2E with Dhanush] Fingerprint [Fingerprint] : 2aa758a9b00692f5b006d687ff60fb63f619fb884aaf80392ef4fe8006db34fb
[E2E] Secure session with Dhanush established!
>
[E2E] Key-exchange initiated by Dhanush.
[E2E with Dhanush] Fingerprint [Fingerprint] : f5644e5f70852a58aa2131ba67f3947d327d00a52ab12f1a29954856c7d26bbb
[E2E] Secure session with Dhanush established!
> Hello?
```

Fig 5.1 Client 1's log showcasing rotation

```
student@cs6008-c2:~/projects/cs6008-secure-chat-app/phase5$ ./client.exe 192.168.56.102 5000
Usage: login <username>

> login Dhanush
Login successful
> /chat Harshit
Current chat: Harshit
> /e2e Harshit
[E2E] Handshake initiated with Harshit. Waiting for reply...
>
[E2E] Reply received from Harshit.
[E2E with Harshit] Fingerprint [Fingerprint] : 6d589cfbc7058c7e0fc8edc79147ca6e80905eb5866c56d47d03c52abf6b901d
[E2E] Secure session with Harshit established!
> Hello! How are you doing??
[E2E] Sending encrypted message to Harshit
>
[REKEY] Sent refresh keys request to Harshit at 2026-09-04 21:58:46
>
[E2E] Reply received from Harshit.
[E2E with Harshit] Fingerprint [Fingerprint] : a47e43c65b3f6c6ff675b901c5d6975a3de45eabc525d88bf5efc7b2a3f3d3fd
[E2E] Secure session with Harshit established!
>
[REKEY] Sent refresh keys request to Harshit at 2026-09-04 21:59:52
>
[E2E] Reply received from Harshit.
[E2E with Harshit] Fingerprint [Fingerprint] : 2aa758a9b00692f5b006d687ff60fb63f619fb884aaf80392ef4fe8006db34fb
[E2E] Secure session with Harshit established!
>
[REKEY] Sent refresh keys request to Harshit at 2026-09-04 22:00:54
>
[E2E] Reply received from Harshit.
[E2E with Harshit] Fingerprint [Fingerprint] : f5644e5f70852a58aa2131ba67f3947d327d00a52ab12f1a29954856c7d26bbb
[E2E] Secure session with Harshit established!
>
[E2E][Harshit] Hello?
>
```

Fig 5.2 Client 2's log showcasing rotation

2. Post-Rotation Message Delivery

Immediately following a key rotation, a chat message was transmitted and verified to decrypt properly using the newly established key.

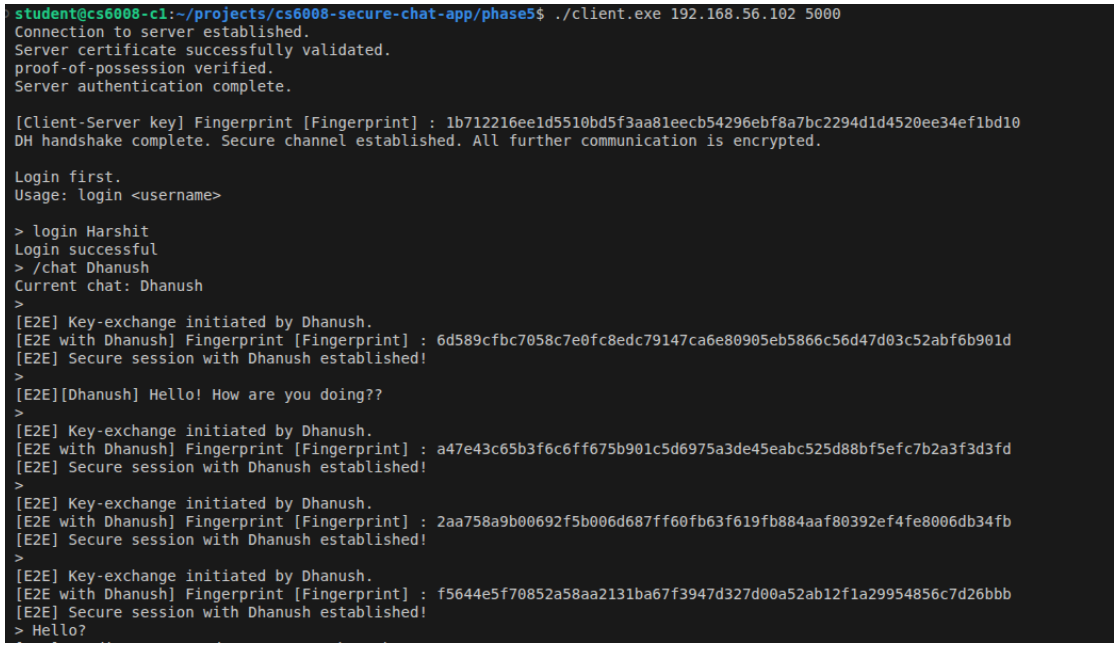

Fig 5.3 Client 1's log showcasing message being sent successfully just after rotation

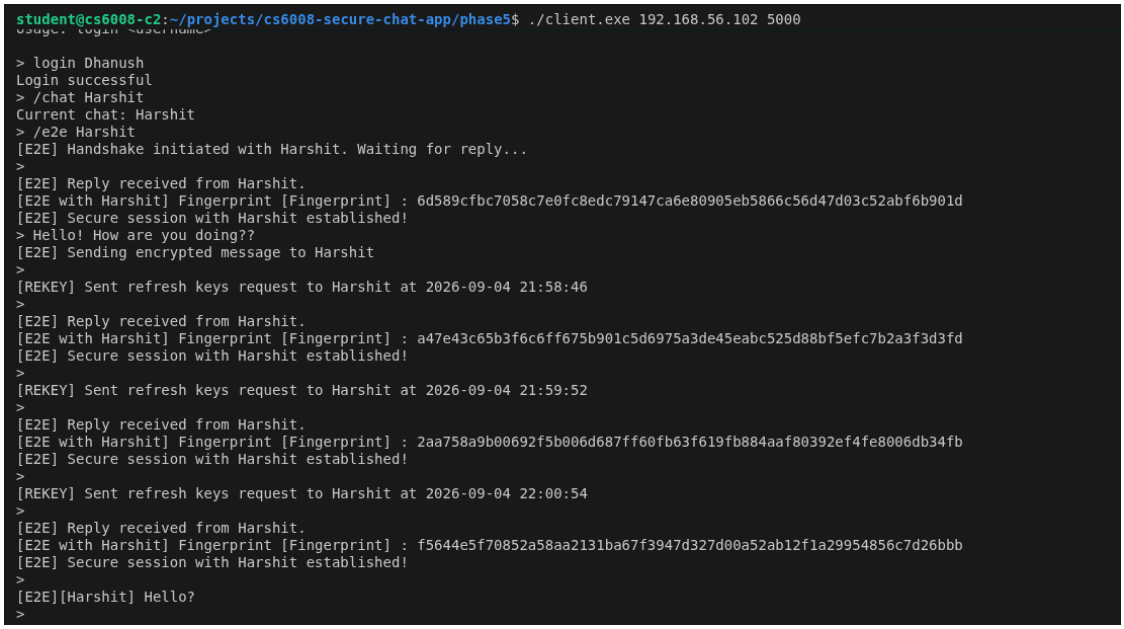

Fig 5.4 Client 2's logs showcasing message being received successfully just after rotation

Detailed Analysis: Key Compromise Impact (Phase 4 vs Phase 5)

Scenario:

An attacker records all encrypted network traffic and later compromises a single active symmetric key at time T.

Phase 4 (Static End-to-End Encryption):

What the Attacker CAN do:

- Decrypt the ENTIRE past conversation history: Because Phase 4 uses a single E2E key for the entire session, this one key decrypts all recorded messages from the beginning of the chat up to time T.
- Decrypt all FUTURE messages: As long as the clients keep chatting without restarting, future messages still use the same key, allowing passive real-time eavesdropping.
- Forge and inject messages: The attacker can craft arbitrary new messages with valid AES-GCM tags impersonating either client.

What the Attacker CANNOT do:

- Decrypt messages from sessions with other users (each peer has a distinct key).
- Decrypt traffic if they did not record the packets from the network.

Phase 5 (Forward Secrecy via 60-Second Ephemeral Rekeying):

What the Attacker CAN do:

- Decrypt ONLY messages within that specific 60-second window: The attacker can only read messages encrypted under that single compromised key.
- Forge or modify messages ONLY during that 60-second window: Tampering is restricted to that active interval only.

What the Attacker CANNOT do:

- CANNOT decrypt PAST messages (Forward Secrecy): All earlier conversation epochs used past ephemeral DH keys that were erased and discarded from memory. Because the old DH private exponents are destroyed, computing past keys is mathematically impossible.
- CANNOT decrypt FUTURE messages after the next rotation (Self-Healing): On the very next 60-second rotation, the clients generate brand-new random DH private exponents. The new key is completely independent of the compromised key, automatically restoring confidentiality and locking the attacker out.

What it buys you:

In Phase 4, a single key leak is catastrophic and compromises the entire conversation history. In Phase 5, the damage of a key compromise is strictly contained to a maximum 60-second window; past communications remain permanently confidential.

For Example:

If Mallory is an active on-path attacker (MITM) and has the key:

She can construct: *aes_gcm_encrypt(stolen_key, "Hey Bob, transfer me $5000")*, wrap it in *__E2E_MSG__*|, and inject it into the TCP connection.

Bob's client will decrypt it with the matching key and show [E2E][Alice] Hey Bob, transfer me $5000. Bob will completely believe Alice sent it.

In Phase 4, Mallory can do this indefinitely.

In Phase 5, Mallory can only do this until the 60-second timer hits and Alice & Bob negotiate a new keypair that Mallory doesn't have.

Github

https://github.com/Dhanush-9/cs6008-secure-chat-app